Hard-Instance Boson14 Stress-Test Pipeline

Brockington-style hard benchmarks for Boson14 hardware

QCi Hardware Team

April 23, 2026

Quantum Computing Inc.

Why a Separate Hard-Instance Pipeline?

- Random planted cliques (`full-benchmark.py`) are **too easy**: $k \gg \sqrt{n} \Rightarrow$ gradient descent finds ω reliably
- To stress-test hardware we need instances that **defeat classical continuous solvers**: SLSQP, L-BFGS, PGD
- Five construction strategies from the literature (Brockington-Culberson, Walteros-Buchanan, Gamarnik-Zadik, Bomze)
- Same downstream tooling: 3 permutation variants per instance, one CSV per variant, Boson14 hardware via `run_boson14.py`

Goal: Identify instances where hardware outperforms (or at least matches) multi-restart gradient methods.

The Five Hard-Instance Strategies

Code	Name	Key Parameter	Hardness Signal
A	Near-threshold	$k \approx k_{\text{mult}} \cdot \log_2 n$	k near detection threshold
B	Dense random	$p \in [0.8,\ 0.9]$	Exponentially many near-optimal cliques
C	Degenerate	$\#\text{planted} = \{2, 3, 5\}$	Multiple competing basins
D	Camouflage	`removal_frac` $\in [0.3, 0.5]$	Suppressed degree on clique vertices
E	Overlap	$\lvert S \cap T \rvert = k-2$	Spurious solutions on $S \cup T$

- **A, C, D, E** have known planted cliques (ground-truth ω is exact)
- **B** has no planted clique — brute-force Bron-Kerbosch finds ω for $n \leq 90$
- Defaults: `-strategies B,D` (hardest for classical continuous solvers in prior sweeps)

Prior Sweep: Classical Solver Hit-Rates (n=90, 30 restarts)

Strategy	Count	Avg ω	SLSQP hit	PGD hit	L-BFGS hit
A near-threshold	9	16.7	73%	70%	0%
B dense random	6	24.3	5%	13%	0%
C degenerate	9	10.3	6%	9%	0%
D camouflage	12	14.0	1%	2%	0%
E overlap	9	12.4	84%	87%	0%

Key observations:

- **B** and **D** are the hardest — **targets for Boson14 stress testing**
- L-BFGS (softmax parameterisation) fails across the board
- **E** is easy for omega recovery but produces spurious supports (Bomze-style issue)

Permutation Variants: Front / End / Random

For each instance we emit three isomorphic copies of the graph, differing only in how vertex labels map to matrix indices:

- **front:** reference clique at indices $[1, 2, \ldots, k]$
- **end:** reference clique at indices $[n - k + 1, \ldots, n]$
- **random:** reference clique at k random indices

What this tests: the hardware should recover the same ω regardless of where in the adjacency matrix the clique sits. Any amplitude-dependent or position-dependent bias becomes visible.

Strategy B special case: for $n \leq 90$ we brute-force a max clique first and treat it as the "reference" for front/end targeting. For $n > 90$ we emit only random variants (brute-force becomes intractable).

Pipeline: Four Phases, Stoppable

Each phase writes a `phase_{name}.done` marker for idempotent resume.

Phase	What it does	Hardware?
`generate`	Build G(n,p) + hard-instance modification	No
`variants`	Emit 3 permutation variants + CSV + meta	No
`verify`	Brute-force ω + theoretical solutions NPZ	No
`hardware`	Call `run_boson14.py` per variant $\times$ amplitude	Yes

```python
# Stop after CSVs are generated (no hardware needed)
python pipeline.py --strategies D --n 50 --k 12 --seeds 42 \
    --phase variants --verify

# Full pipeline on hardware
python pipeline.py --strategies B,D --n 90 --seeds 42 \
    --amplitudes 300,400,500,600

# Resume after an interruption
```

Phase 1 – generate

```python
python pipeline.py --strategies B --n 20 --seeds 42 --phase generate
```

```
==== Strategy B, seed 42 ====
  [dense_random_n20_p0.9_s42] brute-force omega = 11
      (reference clique: [1,2,3,6,7,9,10,11,12,14,15])
  [dense_random_n20_p0.9_s42] generate: saved to .../hard_dense_random_n20_p0.9_s42
```

Produces:

- `base_meta.json` — strategy, n, p, seed, reference clique, $g^\star$
- `base_adjacency.npz` — raw A matrix
- `phase_generate.done`

For strategy B, the brute-force max-clique search runs automatically inside this phase so later phases have a reference clique for permutation.

Phase 2 – variants

```python
python pipeline.py --strategies B --n 20 --seeds 42 \
    --phase variants --skip-existing
```

```
[hard_dense_random_n20_p0.9_s42] variants...
    [front]  sanity check PASSED, targets=[1..11]
    [end]    sanity check PASSED, targets=[10..20]
    [random] sanity check PASSED, targets=[1,4,5,6,8,9,12,14,16,18,20]
```

Produces (per variant):

- `{variant}_boson14.csv` — input for `run_boson14.py` (format: $[C_0 \mid A]$)
- `{variant}_boson14.json` — polynomial form (for Dirac-3 compatibility)
- `{variant}_meta.json` — forward/inverse permutation maps

Sanity check (hard failure on mismatch): the submatrix at the target positions must be a complete subgraph (all-ones minus diagonal).

Phase 3 – verify (optional)

```python
python pipeline.py --strategies D --n 50 --k 12 --seeds 42 \
    --phase verify --verify --skip-existing
```

For each variant:

- Rebuild the graph from the CSV
- Enumerate all max cliques via `networkx.find_cliques` (Bron-Kerbosch, lazy)
- Save theoretical optimal y vectors to `{variant}_solutions.npz` (one row per max clique)
- Update `{variant}_meta.json` with `omega_verified` and `num_max_cliques`

Cost: Exponential in n (Bron-Kerbosch). Practical for $n \leq 90$; scales up to a 120 s timeout beyond which counts become approximate.

Phase 4 – hardware

```python
python pipeline.py --strategies D --n 50 --k 12 --seeds 42 \
    --amplitudes 300,400,500,600 \
    -R 100 -s 1000 -l 256 -d 86 -w 1 -b 28 -solps 0
```

For each variant × amplitude:

1. `subprocess.run([python, run_boson14.py, csv, ...flags], cwd=variant_dir)`
2. Measure wall-clock time from start of subprocess to completion
3. Post-process NPZ: inject `amplitude`, `device_time_wrapper_s`, and all hardware flags
4. Rename `results__*.npz` / `plot__*.png` to add `_a{amp}` suffix
5. Append an entry to instance-level `timing_summary.json`

Critical: `run_boson14.py` is called **unchanged** as a subprocess; we only wire up I/O around it.

Hardware Flags (Pass-Through to `run_boson14.py`)

Flag	Default	Description
`-amplitudes`	`300,400,500,600`	One hardware run per variant per amplitude
`-R` / `-r`	100	Sum constraint
`-s` / `-num-samples`	200	Hardware samples
`-l` / `-num-loops`	200	Computation loops
`-d` / `-delay`	86	Response delay
`-w` / `-pulse-width`	1	—
`-b` / `-distance-between-pulses`	28	—
`-solps`	0	Matplotlib block flag (0 in pipeline mode)

Each `results__*.npz` gets **all** flag values written into it, so later analysis can read the hardware config from the NPZ alone (no filename parsing).

Device-Time Tracking

Measured: wall-clock time from subprocess start to subprocess exit (`device_time_wrapper_s`). Includes subprocess spawn + hardware compute + result I/O.

Per-NPZ: one field in each `results__*.npz` **Per-instance:** `timing_summary.json` at the instance root

```
{
  "entries": [
    {"variant": "front", "amplitude": 300, "device_time_wrapper_s": 57.2,
     "timestamp": "2026-04-23T10:21:42Z",
     "result_filename": "results__front_boson14__20260423_102143_a300.npz",
     "hardware_flags": {"R": 100, "num_samples": 1000, "num_loops": 256, ...}},
    {"variant": "front", "amplitude": 600, "device_time_wrapper_s": 58.1, ...}
  ],
  "totals": {"num_runs": 12, "total_device_time_s": 695.8}
}
```

Output Layout (One Instance)

```
output/hard_D_camouflage_n90_k12_rf0.4_p0.5_s42/
  base_meta.json                # strategy, n, k, p, seed, reference_nodes, g_star
  base_adjacency.npz            # raw A matrix
  timing_summary.json           # per-variant per-amplitude runtimes
  phase_generate.done           # idempotency markers
  phase_variants.done
  phase_verify.done
  phase_hardware.done
  front/
    front_meta.json             # variant metadata + perm maps
    front_boson14.csv           # [C_zeros | A_variant] - input for run_boson14.py
    front_boson14.json          # polynomial form
    front_solutions.npz         # theoretical optimal y (from --verify)
    results__front_boson14__20260423_102143_a300.npz
    plot__front_boson14__20260423_102143_a300.png
    results__front_boson14__20260423_102143_a600.npz
    plot__front_boson14__20260423_102143_a600.png
  end/
```

CLI Quick Reference

```python
# Strategy B and D defaults, single seed, four amplitudes
python pipeline.py --n 90 --seeds 42 --amplitudes 300,400,500,600

# All five strategies, three seeds, custom k
python pipeline.py --strategies A,B,C,D,E --n 60 --k 15 \
    --seeds 42,123,7 --amplitudes 600

# Offline only (no hardware): generate CSVs for later use
python pipeline.py --strategies D --n 50 --k 12 --seeds 42 \
    --phase variants --verify

# Resume the hardware phase after a crash
python pipeline.py --strategies B,D --n 90 --seeds 42 \
    --amplitudes 300,400,500,600 --skip-existing

# Dry-run: print commands without calling hardware
python pipeline.py --strategies D --n 20 --k 6 --seeds 42 \
    --amplitudes 300 --dry-run
```

Self-Containment

Directory: `boson14/hard-instances-benchmarks/`

- `pipeline.py` — main CLI, four phases
- `utils.py` — variant generation + I/O helpers
- `_internal/generators.py` — strategy definitions (copied from `hard-instances/`)
- `_internal/bruteforce.py` — exact max-clique via Bron-Kerbosch
- `README.md` — full usage docs
- `manual.tex` — this document

External dependencies:

- `boson14_bench` (parent package) for permutation + polynomial helpers
- `run_boson14.py` (parent directory) — invoked as subprocess, never modified

The subfolder can be moved anywhere on the filesystem as long as a compatible

Design Principles

- **Don't touch** `run_boson14.py` — wrap its CWD-relative I/O via `cwd=variant_dir` and post-process the NPZ
- **Phase-based** — each phase is independently re-runnable; markers make resume cheap
- **Sanity-check permutations** — the all-ones-minus-diagonal submatrix test catches 0-based/1-based bugs at the source
- **Single authoritative timing record** — `timing_summary.json` is upserted, never appended blindly
- **Brute-force gated by** n — $n \leq 90$ gets full verification; $n > 90$ gracefully degrades to random-only variants for strategy B
- **Complete hardware provenance** — every result NPZ carries its own full hardware flag set; no external context needed to interpret a lone file

Future Work

- **Report scripts:** extend `boson14/scripts/extract_results.py` to pick up `hard_*` folders (pattern: `benchmark_*|hard_*`)
- **Comparative analysis:** overlay Boson14 amplitude sweeps on the same axes as the prior classical-solver hit-rate distributions
- **Strategy parameter grids:** automate the parameter sweep for D (vary `removal_frac` $\in \{0.3, 0.4, 0.5\}$) and A (vary `k_multiplier`)
- **Failure-mode analysis:** cluster samples whose support is *not* a clique (Bomze repair overlap with existing `boson14_repair.py`)
- **Bomze regularisation:** add `-bomze` flag that writes $J = -0.5(A + 0.5I)$ instead of $-0.5A$ to eliminate spurious local optima

References

- Motzkin, Straus (1965). "Maxima for graphs and a new proof of a theorem of Turán." *Canadian J. Math. 17*, 533–540.
- Bomze (1997). "Evolution towards the maximum clique." *J. Global Opt. 10*, 143–164.
- Gibbons, Hearn, Pardalos, Ramana (1997). "Continuous characterizations of the maximum clique problem." *Math. Oper. Res. 22(3)*, 754–768.
- Brockington, Culberson (1993). DIMACS 2nd Implementation Challenge instance generators.
- Gamarnik, Zadik (2019). "The Landscape of the Planted Clique Problem: Dense subgraphs and the Overlap Gap Property." arXiv:1904.07174.
- Walteros, Buchanan (2020). "Why Is Maximum Clique Often Easy in Practice?" *Operations Research*.
- Chen, Moitra, Rohatgi (2024). "Finding Planted Cliques Using Gradient Descent." *SIMODS*.